Limitações do modelo de tradução Helsinki-NLP/opus-mt-en-fr

- A qualidade depende do quanto o assunto aparece no OPUS: termos técnicos, nomes próprios e áreas pouco representadas podem sair errados ou ficar sem tradução. O modelo traduz frase a frase, sem enxergar o resto do documento. Por isso pode errar e gerar ambiguidades.

- A tradução é feita token a token, ou seja, o tempo vai crescendo de acordo com o tamanho do texto.
- Em relação ao custo, o modelo é aberto e roda localmente. Então não tem cobrança por chamada.

- E por fim, o modelo pode traduzir expressões de idiomas ao pé da letra. Gírias novas provavelmente não serão conhecidas

Chatbot com Fake LLM

O fluxo basicamente funciona assim: o cliente envia a mensagem em JSON para a rota POST /chat. O FastAPI valida o JSON com Pydantic e procura a mensagem no dicionário de perguntas pré-definidas. O LangChain monta o prompt com o PromptTemplate e o envia ao FakeListLLM, que devolve a resposta definida. A API retorna a pergunta e a resposta em JSON.

A importância de usar os fakes LLM para testes rápidos são que:

- Não há custo, pode-se testar a vontade sem gasto.
- Os teste rodam localmente na máquina.
- Respostas são previsíveis, uma vez que a mesma entrada gera a mesma saída.
- A uma troca simples do LLM fake para o real.

Limitações do LangChain com a API da OpenAI

- Em termos de latência, cada tradução é uma chamada pela internet até os servidores da OpenAI, e o tempo de ida e volta soma-se ao tempo de processamento do modelo. Esse tempo varia com a carga da OpenAI e acaba fudgindo do controle da aplicação. Os modelos da linha GPT-5 fazem um raciocínio interno antes de responder, o que acrescenta tempo.
- A OpenAI limita o número de requisições da API e de tokens também. Há limite de tamanho do texto de entrada e saída por chamada.
- Como é um LLM generalista, costuma lidar melhor com contexto, expressões idiomáticas e tom. Aceita instruções específicas mas pode acabar “delirando” e fazendo coisas não solicitadas.

Limitações do LangChain com o modelo de tradução da HuggingFace

- Em termos de desempenho e latência, se rodadi no próprio CPU, a tradução dos textos longos fica mais lenta. É também necessária a divisão dos textos.
- Além disso, temos consumo de recusros como Pytorch e manter o modelo na memória do computador, o que custa processamento. Cada API carrega um modelo, pode às vezes ser necessário o uso de uma GPU.
- O LangChain faz inferência, não tem ajuste fino. Para treinamento, é necessário uso do transformers diretamente, dados de tradução em pares e etc.
- O LangChain oferece uma camada de abstração que não ajuda muito com um único modelo, mas tem a padronização da interface (trocar o modelo local pela openAI basta ajustar uma linha). Já a API da Hugging, a execução local não paga chamada da API não envia dados para fora, mas existe manter o servidor e a memória.

Comparação entre FastAPI puro e LangChain

!!! Tabela feita por IA, eu adorei a forma como a IA colocou simples em uma tabela as comparações entre a fastapi e o Langchain, por isso deixei direto aqui abaixo o print.

Abordagem	Facilidade de uso e configuração	Latência e desempenho	Flexibilidade para diferentes modelos	Custo e escalabilidade	Protótipo ou produção
FastAPI puro (`transformers` direto)	Poucas dependências e código direto: o pipeline do `transformers` é chamado na rota. Exige instalar o PyTorch e baixar o modelo.	Sem camadas extras e sem rede na chamada. A latência depende do hardware local; na CPU, cresce com o tamanho do texto.	Baixa: cada modelo tem seu jeito de ser chamado e de devolver a resposta, e trocar de modelo exige mudar o código.	Sem custo por chamada. Escalar exige mais servidores ou GPU, cada um com uma cópia do modelo.	Bom para protótipos com um único modelo e para produção quando o modelo é fixo e há infraestrutura própria.
LangChain (OpenAI ou HuggingFace)	Mais bibliotecas e conceitos (prompt, cadeia, parser), mas a mesma estrutura serve para qualquer modelo. Com OpenAI, não há download, só a chave de API.	Acrescenta uma camada pequena. Com OpenAI, soma o tempo de rede e depende da carga do fornecedor. Com HuggingFace local, o desempenho é o do `transformers`.	Alta: a interface é a mesma para OpenAI, HuggingFace e Fake LLM, e trocar de modelo muda uma linha.	Com OpenAI, escala sem infraestrutura própria, mas paga por token e tem limites de uso. Com HuggingFace, valem os custos de infraestrutura do FastAPI puro.	Muito bom para protótipos, porque permite trocar modelos e testar com Fake LLM. Em produção, ajuda quando há cadeias, vários modelos ou troca de fornecedor, ao custo de mais uma dependência para manter.

Código no github: https://github.com/leonardossantosrj/infnet_tp2